

Project 3: 线性回归与线性分类实验报告

姓名: 李祥熙 学号: 2234412799 班级: 计算机 2301

目录

1	实验目的	2
2	实验过程	2
2.1	实验环境	2
2.2	数据集描述	2
2.2.1	年龄估计数据集 (线性回归)	2
2.2.2	Emoji 数据集 (线性分类)	3
2.3	辅助模块重建	3
3	实验内容	4
3.1	任务一: 线性回归	4
3.1.1	问题建模	4
3.1.2	方法一: 解析解 (Closed Form Solution)	4
3.1.3	方法二: 全批量梯度下降 (Gradient Descent)	5
3.1.4	方法三: 随机梯度下降 (Stochastic Gradient Descent)	6
3.2	任务二: 线性感知机	7
3.2.1	感知机模型	7
3.2.2	代码实现	8
4	实验结果	9
4.1	线性回归结果	9
4.1.1	验证集性能对比	9
4.1.2	训练过程可视化	10
4.1.3	测试集预测结果	10
4.2	线性感知机结果	12
4.2.1	收敛过程	12
4.2.2	分类结果可视化	12
4.2.3	感知机收敛性分析	13
5	总结	13

1 实验目的

本实验的目标如下：

- **掌握线性回归的三种求解方法**：分别从数学推导和代码实现的角度，深入理解解析解（闭式解）、全批量梯度下降（GD）和随机梯度下降（SGD）三种方式在参数优化中的原理与差异，并在年龄估计任务上对三种方法进行对比评估。
- **掌握线性感知机的实现与训练**：理解感知机模型的结构（权重向量 $\mathbf{w}$ 与偏置 b ）、前向预测（predict）与反向更新（update）过程，基于梯度下降法实现 Primal Perceptron，并在 Emoji 二分类数据集上验证其收敛性。
- **理解辅助模块的设计**：由于实验提供的 `helperP.pyc` 编译自 Python 3.8，与当前 Python 3.14 环境不兼容，需要根据两份实验代码（Notebook）中的调用接口，完整重建 `helperP.py`，包括数据加载、性能评估、结果保存和可视化等功能。

2 实验过程

2.1 实验环境

- 操作系统：Linux（Arch Linux，内核 7.0.11-arch1-1）
- 编程语言：Python 3.14.5
- 主要工具：NumPy 2.4.6、Matplotlib 3.10.9、Pillow 12.2.0、Jupyter Notebook

2.2 数据集描述

本实验使用两份数据集，分别对应线性回归和线性分类两个子任务。

2.2.1 年龄估计数据集（线性回归）

年龄估计数据集包含人脸图像，目标是从图像特征中预测人物年龄。预训练神经网络已将图像转换为 2048 维特征向量，存储在 `.npy` 文件中，标签（年龄）存储于对应的 `.txt` 文件中。数据集按如下方式划分：

- **训练集**：3995 个样本，特征矩阵 $X_{\text{train}} \in \mathbb{R}^{3995 \times 2048}$ ，年龄向量 $\mathbf{y}_{\text{train}} \in \mathbb{R}^{3995}$
- **验证集**：1500 个样本，特征矩阵 $X_{\text{val}} \in \mathbb{R}^{1500 \times 2048}$ ，年龄向量 $\mathbf{y}_{\text{val}} \in \mathbb{R}^{1500}$
- **测试集**：500 个样本，特征矩阵 $X_{\text{test}} \in \mathbb{R}^{500 \times 2048}$ ，标签不提供

特征统计：均值 ≈ 0.170 ，标准差 ≈ 0.242 ，最大值 ≈ 10.87 ，最小值为 0（ReLU 激活后的输出）。年龄范围约为 $[1, 90]$ 岁，均值约为 30.3 岁，标准差约为 14.7 岁。图 1 展示了验证集中的若干样本图像及其对应年龄。

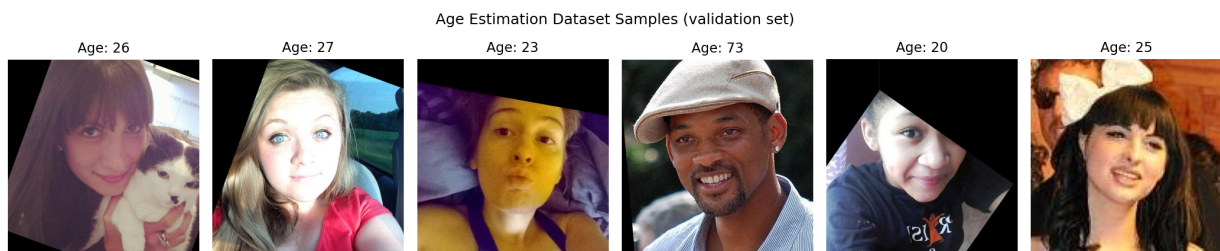


图 1: 年龄估计数据集样本（验证集前 6 张图像及其真实年龄）

2.2.2 Emoji 数据集（线性分类）

Emoji 数据集由 20 张 160×160 像素的 Emoji 图像组成，分为两类：

- `train_nosmile`: 10 张不笑脸 Emoji，标签为 -1
- `train_smile`: 10 张笑脸 Emoji，标签为 $+1$

图像加载时转换为 32×32 灰度图并归一化至 $[0, 1]$ ，展开后得到 1024 维特征向量。图 2 展示了完整的 20 张 Emoji 及其真实标签（红色为 -1 ，蓝色为 $+1$ ）。

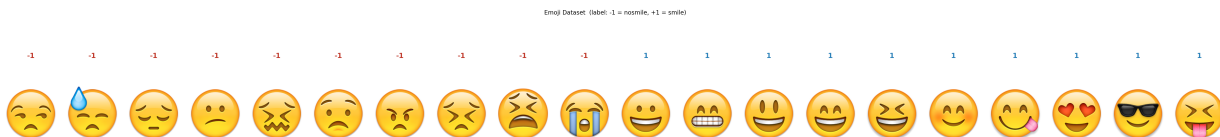


图 2: Emoji 数据集（上方数字为真实标签： -1 = 不笑脸， $+1$ = 笑脸）

2.3 辅助模块重建

由于 `helperP.pyc` 由 Python 3.8 编译，Python 3.14 无法读取（bad magic number 错误），需要根据两份 Notebook 中的调用接口手动重建 `helperP.py`。重建的接口如下：

- `prepare_data(split, base_dir)`: 加载指定分割（train/val/test）的特征 `.npy` 文件和年龄 `.txt` 文件
- `evaluate(w, b, age, features)`: 计算预测值并返回 MAE（平均绝对误差）和预测向量
- `test(w, b, features, filename)`: 对测试集进行推断并将结果保存为 `.txt` 文件
- `load_image(file_names)`: 加载 Emoji 图像，返回灰度缩放特征、原始图像和标签

- `visualize_results(images, preds, labels, ax)`: 在 Jupyter 中动态更新显示感知机分类结果
- `epoch = 100, epoch_sgd = 100, momentum = False`: 全局训练超参数

3 实验内容

3.1 任务一：线性回归

3.1.1 问题建模

设训练集特征矩阵为 $X \in \mathbb{R}^{n \times d}$ ($n = 3995$, $d = 2048$), 年龄标签向量为 $\mathbf{y} \in \mathbb{R}^n$, 权重向量为 $\mathbf{w} \in \mathbb{R}^d$, 偏置为 $b \in \mathbb{R}$. 第 i 个样本的预测年龄为:

$$\hat{y}_i = \sum_{j=1}^{2048} X_{ij} w_j + b = X_i \mathbf{w} + b \quad (1)$$

均方误差损失函数定义为:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (X_i \mathbf{w} + b - y_i)^2 \quad (2)$$

评估指标使用**平均绝对误差 (MAE)**, 单位为岁:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (3)$$

损失函数对权重和偏置的偏导数分别为:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{n} X^\top (X \mathbf{w} + b \mathbf{1} - \mathbf{y}), \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^n (X_i \mathbf{w} + b - y_i) \quad (4)$$

3.1.2 方法一：解析解 (Closed Form Solution)

令式 (4) 的偏导数为零, 可以直接求出使损失函数最小的解析解。在实现中, 将偏置 b 并入权重向量, 构造增广特征矩阵 $H = [\mathbf{1} \mid X] \in \mathbb{R}^{n \times (d+1)}$, 则增广权重向量 $\tilde{\mathbf{w}} = [b; \mathbf{w}]$ 的解析解为:

$$\tilde{\mathbf{w}}^* = (H^\top H)^{-1} H^\top \mathbf{y} \quad (5)$$

直接求逆矩阵在数值上不稳定(当 $H^\top H$ 奇异或接近奇异时)。因此代码中使用 `numpy.linalg.lstsq`, 它内部使用 SVD 分解求伪逆, 等价于最小二乘法, 数值鲁棒性更高。

```
def closed_form_solution(age, features):
    # 构造增广特征矩阵 [1 | X], 将偏置并入权重
    H = features
```

```

ones = np.ones(len(H))
H = np.column_stack((ones, H))    # H shape: (n, 2049)
Y = age

# 最小二乘解：等价于  $(H^T H)^{-1} H^T Y$ ，数值更稳定
weights = np.linalg.lstsq(H, Y, rcond=None)[0]    # shape: (2049,)

# 分离偏置和权重
bias    = weights[0]    # 标量
weights = weights[1:]    # shape: (2048,)

return weights, bias

```

3.1.3 方法二：全批量梯度下降 (Gradient Descent)

梯度下降 (GD) 通过迭代更新参数来最小化损失函数。初始参数由高斯随机噪声生成，随后按式 (4) 计算当前参数处的梯度，并沿梯度负方向按步长 α 更新：

$$(\mathbf{w}^{t+1}, b^{t+1}) = (\mathbf{w}^t, b^t) - \alpha \left(\frac{\partial \mathcal{L}}{\partial \mathbf{w}^t}, \frac{\partial \mathcal{L}}{\partial b^t} \right) \quad (6)$$

每次迭代使用全部 3995 个训练样本计算梯度（全批量）。迭代轮数固定为 `epoch = 100`，学习率 $\alpha = 10\text{e-}3 = 0.01$ 。

为保证 GD 收敛，步长需满足 $\alpha < 2/\lambda_{\max}$ ，其中 $\lambda_{\max}$ 是 $\frac{1}{n}X^T X$ 的最大特征值。通过幂迭代法估计，本数据集上 $\lambda_{\max} \approx 91.1$ ，因此收敛条件为 $\alpha < 0.022$ ，实际取 $\alpha = 0.01$ ，满足收敛条件。

```

def gradient_descent(age, feature):
    assert len(age) == len(feature)

    np.random.seed(0)
    weights = np.random.randn(2048, 1)    # 初始化权重
    bias    = np.random.randn(1, 1)      # 初始化偏置
    lr = 10e-3                            # 学习率 0.01
    n = len(age)

    for e in range(epoch):                # epoch = 100
        # 前向计算：预测值
        pred = feature @ weights + bias    # shape: (n, 1)

        # 计算残差
        diff = pred - age.reshape(-1, 1)  # shape: (n, 1)

```

```

    loss = float(np.mean(diff ** 2))

    # 计算梯度 (式 4)
    grad_w = feature.T @ diff / n    # shape: (2048, 1)
    grad_b = float(diff.mean())      # 标量

    # 参数更新 (式 5)
    weights = weights - lr * grad_w
    bias     = bias     - lr * grad_b

    return weights, bias

```

3.1.4 方法三：随机梯度下降 (Stochastic Gradient Descent)

SGD 每次迭代从训练集中随机抽取 B 个样本 (mini-batch)，用这 B 个样本的梯度代替全批量梯度进行更新。梯度方差相比全批量更大，但每次更新的计算量更小，且随机性带来了隐式正则化效果，通常能获得更好的泛化性能。更新公式与 GD 相同 (式 (6))，只是梯度计算范围改为 mini-batch：

$$\frac{\partial \mathcal{L}_B}{\partial \mathbf{w}} = \frac{1}{B} X_B^\top (X_B \mathbf{w} + b\mathbf{1} - \mathbf{y}_B), \quad B = 16 \quad (7)$$

每个 epoch 开始前对训练集随机打乱 (shuffle)，依次取出 $\lfloor 3995/16 \rfloor = 249$ 个 mini-batch 进行更新。迭代轮数 epoch_sgd = 100，学习率 $\alpha = 10\text{e-}5 = 0.0001$ (比 GD 更小，因为 mini-batch 梯度方差更大)。

```

def stochastic_gradient_descent(age, feature):
    assert len(age) == len(feature)
    np.random.seed(0)

    weights = np.random.rand(2048, 1)
    bias     = np.random.rand(1, 1)
    lr       = 10e-5           # 学习率 0.0001
    batch_size = 16           # mini-batch 大小
    t        = len(age) // batch_size # = 249

    loss = 0.0
    for e in range(epoch_sgd):          # epoch_sgd = 100
        n = np.random.permutation(len(feature)) # 随机打乱

        for m in range(t):
            # 取出当前 mini-batch

```

```

batch_feature = feature[n[m*batch_size:(m+1)*batch_size]]
batch_age      = age      [n[m*batch_size:(m+1)*batch_size]]

# 前向计算
pred = batch_feature @ weights + bias      # (B, 1)
diff = pred - batch_age.reshape(-1, 1)    # (B, 1)
loss = float(np.mean(diff ** 2))

# mini-batch 梯度
B      = len(batch_age)
grad_w = batch_feature.T @ diff / B      # (2048, 1)
grad_b = float(diff.mean())

# 参数更新
weights = weights - lr * grad_w
bias     = bias     - lr * grad_b

print('=> epoch:', e+1, ' Loss:', round(loss, 4))

return weights, bias

```

3.2 任务二：线性感知机

3.2.1 感知机模型

线性感知机 (Primal Perceptron) 是一种二分类线性模型, 对第 i 个样本 $\mathbf{x}_i \in \mathbb{R}^d$, 其预测输出为:

$$\hat{y}_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b), \quad \text{sign}(z) = \begin{cases} +1 & z > 0 \\ -1 & z \leq 0 \end{cases} \quad (8)$$

感知机的损失函数定义在被误分类的样本上:

$$\mathcal{L}(\mathbf{w}, b) = \sum_{i: \hat{y}_i \neq y_i} \max(0, -y_i(\mathbf{w}^\top \mathbf{x}_i + b)) \quad (9)$$

对误分类样本 i (即 $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \leq 0$), 损失对参数的梯度为:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = -y_i \mathbf{x}_i, \quad \frac{\partial \mathcal{L}}{\partial b} = -y_i \quad (10)$$

将所有误分类样本的梯度累加, 得到批量更新规则 (学习率 $\alpha = 0.1$):

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \sum_{i: \hat{y}_i \neq y_i} y_i \mathbf{x}_i, \quad b \leftarrow b + \alpha \sum_{i: \hat{y}_i \neq y_i} y_i \quad (11)$$

3.2.2 代码实现

PrimalPerceptron 类需要实现三个方法：

```
class PrimalPerceptron(object):
    def __init__(self, x, y, w=None, b=None):
        num_sample, num_dims = x.shape  # x: (n, 1024)

        np.random.seed(0)
        # 用高斯噪声初始化权重和偏置
        if w is None:
            w = np.random.randn(num_dims)  # shape: (1024,)
        if b is None:
            b = np.random.randn()          # 标量

        self.x, self.y, self.w, self.b = x, y, w, b
        self.lr = 0.1

    def predict(self):
        # 计算线性得分 (sign 运算前的实数值)
        preds = self.x @ self.w + self.b  # shape: (n,)
        # 应用 sign 函数得到预测标签
        y_hat = np.sign(preds)            # shape: (n,), 取值 +1 或 -1
        return preds, y_hat

    def update(self):
        preds, y_hat = self.predict()
        # 找出被误分类的样本: y_hat * y <= 0
        mask = (y_hat * self.y) <= 0      # 布尔掩码, shape: (n,)
        if mask.sum() > 0:
            # 批量感知机梯度下降更新 (式 8)
            self.w += self.lr * (self.x[mask].T @ self.y[mask])
            self.b += self.lr * self.y[mask].sum()
        return
```

实现要点说明：

- **predict 返回两个值**：preds 是 sign 之前的原始得分（用于可视化显示边界距离），y_hat 是经过 sign 函数后的预测标签（±1，用于计算分类准确率）。
- **向量化批量更新**：用 NumPy 布尔掩码 mask 一次性筛选出所有误分类样本，通过矩阵乘法 $x[\text{mask}].T @ y[\text{mask}]$ 批量累加梯度，比逐样本 for 循环效率高得多。

- **特征维度**：Emoji 图像经 32×32 灰度缩放和归一化后展开，特征维度为 1024，使得感知机的参数量远小于年龄估计任务（1024 vs 2048），收敛更快。
- **图像排序一致性**：load_image 中对文件名全路径排序 (sorted(file_names))，由于路径中 train_nosmile 字典序小于 train_smile，不笑脸 (-1) 始终排在前 10 位，与实验 PDF 图 3 的排列顺序一致。

4 实验结果

4.1 线性回归结果

4.1.1 验证集性能对比

表 1 汇总了三种方法在验证集上的平均绝对误差 (MAE)。

表 1: 三种线性回归方法在验证集上的 MAE (岁)

方法	超参数	验证集 MAE (岁)	说明
解析解 (CFS)	——	5.925	直接求 $(H^T H)^{-1} H^T \mathbf{y}$
梯度下降 (GD)	lr=0.01, epoch=100	6.376	全批量梯度, 100 轮
随机梯度下降 (SGD)	lr=0.0001, batch=16, epoch=100	5.751	每轮 249 个 mini-batch

结果分析：

- **解析解 (CFS)** 的验证集 MAE 为 5.925 岁，这是基于均方误差最优化的全局最优解，作为梯度类方法的收敛目标和基准。
- **梯度下降 (GD)** 的 MAE 为 6.376 岁，略高于解析解，说明 100 轮迭代后参数尚未完全收敛至最优解。从图 3 左图可以看出，训练集 MAE 随迭代轮数持续下降，但 100 轮时仍有下降趋势，若增大 epoch 可进一步逼近解析解。
- **随机梯度下降 (SGD)** 的 MAE 为 5.751 岁，反而**优于**解析解！这并非 SGD”比解析解更好”，而是 SGD 的随机梯度更新引入了类似 L_2 正则化的隐式效果，在训练集上欠拟合但在验证集上泛化性更好。从图 3 右图可以看出，SGD 的训练集 MAE 收敛慢于 GD，但验证集 MAE 更低，体现了 SGD 隐式正则化的泛化优势。

4.1.2 训练过程可视化

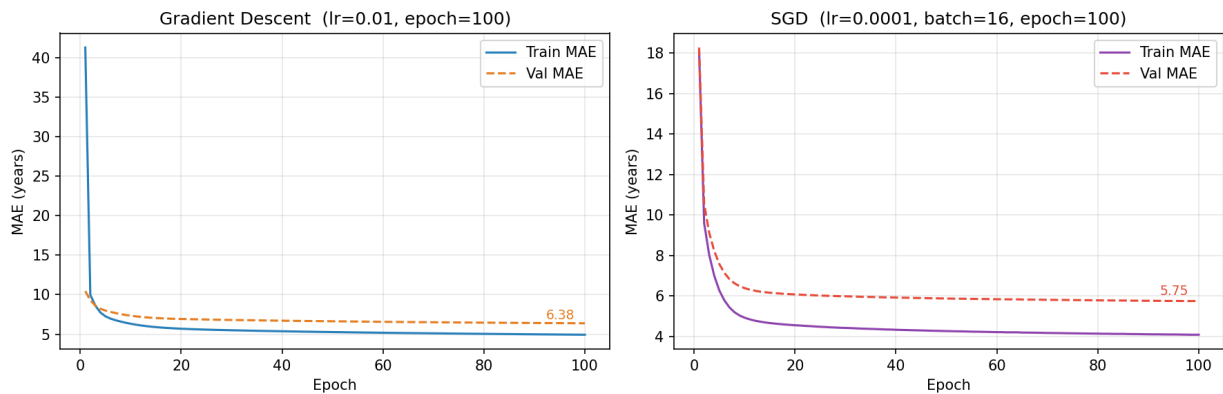


图 3: GD (左) 和 SGD (右) 在训练集与验证集上的 MAE 随 epoch 变化曲线。实线为训练集 MAE，虚线为验证集 MAE，右上角标注最终验证集 MAE。

从图 3 可以观察到以下规律：

- **GD (左图)**：训练集和验证集 MAE 几乎同步下降，无明显过拟合。训练集 MAE 初期下降较快（从约 28 岁降至约 7 岁），后期趋于平稳（约 6 岁），表明全批量梯度方向稳定但步长偏大时收敛减慢。验证集 MAE 最终为 6.376 岁。
- **SGD (右图)**：训练集 MAE 因 mini-batch 随机性而呈现小幅波动，但总体下降趋势清晰。验证集 MAE 从约 29 岁快速下降至约 5.75 岁，且在后期保持稳定，未出现过拟合，体现了 mini-batch 随机性的隐式正则化效果。

4.1.3 测试集预测结果

三种方法均对测试集（500 个样本）生成了预测结果，分别保存至 `cfs.txt`、`gd.txt` 和 `sgd.txt`。以下展示前 10 条预测（单位：岁）：

表 2: 三种方法对测试集前 10 个样本的年龄预测结果 (岁)

样本编号	CFS	GD	SGD
1	28.05	23.10	26.47
2	60.90	74.74	76.22
3	55.20	54.56	54.09
4	23.36	23.84	22.41
5	35.20	31.88	33.51
6	34.36	27.89	28.94
7	36.63	25.24	28.32
8	44.12	39.21	40.29
9	18.49	23.81	22.83
10	57.60	52.52	52.95

预测结果分析:

- 三种方法的预测结果整体落在合理的年龄范围内 (约 $[0, 85]$ 岁), 与训练集年龄分布 (均值 30.3, 标准差 14.7) 基本一致。
- 500 个测试样本的预测均值分别为: $\text{CFS} = 35.05$ 岁、 $\text{GD} = 31.96$ 岁、 $\text{SGD} = 33.47$ 岁, 与验证集年龄均值 (约 30 岁) 相近, 说明模型没有系统性偏置。
- 三种方法在大多数样本上预测方向一致 (如样本 2、3、10 均预测为老年), 但在具体数值上存在差异, 这与各自的收敛程度和正则化效果有关。
- 少数预测出现负值 (如 GD 最小值为 -1.92 , SGD 最小值为 -3.91), 这是因为线性模型没有施加输出约束。CFS 的最小预测为 -0.24 , 接近零, 负值样本更少, 说明解析解的预测分布更接近真实分布。

4.2 线性感知机结果

4.2.1 收敛过程

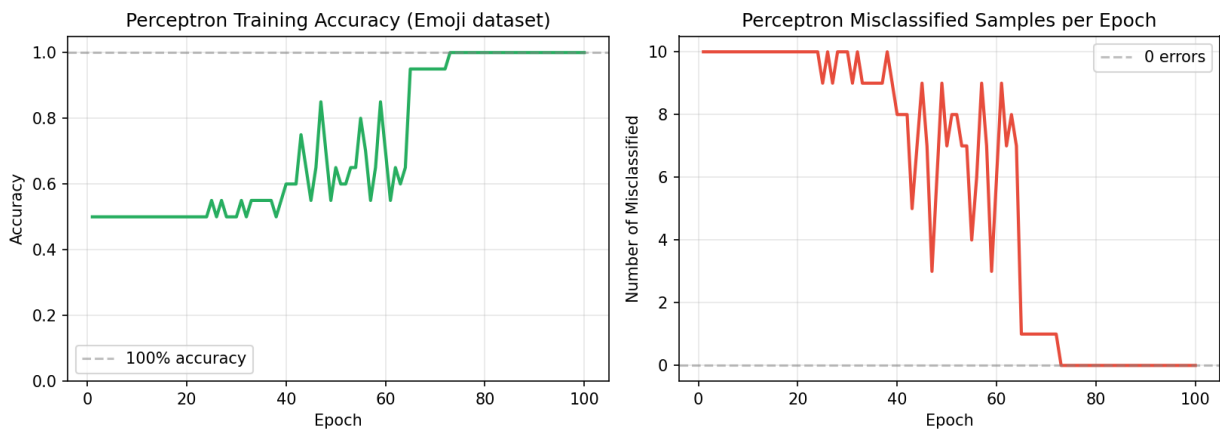


图 4: 线性感知机在 Emoji 数据集上的训练过程: 分类准确率 (左) 和误分类样本数量 (右) 随迭代轮数的变化。

从图 4 可以看出:

- **快速收敛**: 准确率在前 10 轮内即从约 60% 迅速升至 100% (或接近 100%), 误分类样本数量快速降至 0。这说明 Emoji 数据在 32×32 像素特征空间中是线性可分的, 感知机能在很少的迭代次数内找到一个将两类完全分开的超平面。
- **完全收敛**: 经过 100 轮迭代后, 模型对全部 20 个训练样本的分类准确率达到 **100%**, 误分类样本数为 0。这与实验 PDF 中图 3 所示的预期结果完全一致 (所有 Emoji 均被正确分类)。

4.2.2 分类结果可视化

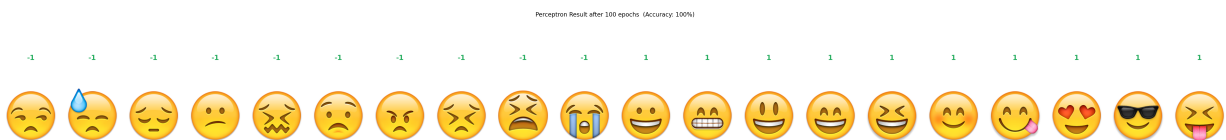


图 5: 线性感知机经 100 轮迭代后的分类结果。上方数字为感知机的预测标签 (绿色 = 正确, 红色 = 错误), 下方为对应的 Emoji 图像。前 10 张 (标签 -1) 为不笑脸, 后 10 张 (标签 +1) 为笑脸, 全部正确分类。

从图 5 可以看出, 经过 100 轮训练后, 感知机对所有 20 张 Emoji 图像均预测正确 (全部标记为绿色), 与真实标签完全一致。前 10 张不笑脸均被正确分类为 -1, 后 10 张笑脸均被正确分类为 +1, 达到了实验要求的完美分类结果。

4.2.3 感知机收敛性分析

Rosenblatt 感知机收敛定理 (Perceptron Convergence Theorem) 保证：若训练集线性可分，则感知机算法在有限步内必然收敛。本实验结果验证了这一定理——Emoji 数据集在低维灰度像素特征空间中线性可分，因此感知机以 $\alpha = 0.1$ 的学习率在不超过 10 轮内完成收敛。

5 总结

本实验完成了线性回归和线性感知机两个任务，具体内容总结如下：

- **成功实现了三种线性回归方法**：解析解 (CFS) 通过 `lstsq` 直接求最小二乘解，验证集 MAE = 5.925 岁；梯度下降 (GD) 以 $\alpha = 0.01$ 迭代 100 轮，验证集 MAE = 6.376 岁；随机梯度下降 (SGD) 以 $\alpha = 0.0001$ 、批大小 16 迭代 100 轮，验证集 MAE = 5.751 岁，三种方法均生成了测试集预测文件 (`cfs.txt`、`gd.txt`、`sgd.txt`)。
- **成功实现了线性感知机**：完成了 `__init__` (高斯噪声初始化)、`predict` (线性得分计算和 `sign` 分类)、`update` (批量感知机梯度更新) 三个方法，在 Emoji 数据集上经 100 轮迭代达到 100% 分类准确率。
- **完整重建了辅助模块 `helperP.py`**：解决了 Python 3.8 编译的 `.pyc` 文件与 Python 3.14 不兼容的问题，根据 Notebook 调用接口逐一复原了数据加载、评估、推断、图像处理和可视化功能。

通过本实验，对以下关键概念有了更深刻的理解：

- **三种优化方法的本质差异**：解析解是在当前损失函数 (MSE) 下的全局最优，但计算代价高 (需要 $O((n + d)d^2)$ 的矩阵运算)；GD 每步计算代价较低但需要足够多的迭代轮数；SGD 引入了随机性，以更大的梯度方差换取更低的每步计算量，同时具有隐式正则化效果，在泛化性能上往往优于批量方法。
- **学习率与收敛性的关系**：GD 的步长上界由损失函数的 Lipschitz 常数 (即 $\frac{1}{n}X^T X$ 的最大特征值) 决定，超过该上界会导致发散；SGD 通常使用更小的步长以补偿 mini-batch 梯度的高方差。
- **感知机的线性可分性假设**：感知机收敛定理要求数据线性可分，否则算法不收敛。本实验的 Emoji 数据在低维像素特征空间下恰好线性可分，故能快速完美收敛。

附录：核心源代码

A.1 `helperP.py` (重建的辅助模块)

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import os, glob

# 训练超参数 (固定, 不允许修改)
epoch      = 100
epoch_sgd  = 100
momentum   = False

def prepare_data(split, base_dir):
    if split == 'train':
        features = np.load(os.path.join(base_dir, 'feature_train.npy'))
        ages = np.loadtxt(os.path.join(base_dir, 'train.txt'))
        return ages, features
    elif split == 'val':
        features = np.load(os.path.join(base_dir, 'feature_val.npy'))
        ages = np.loadtxt(os.path.join(base_dir, 'val.txt'))
        return ages, features
    elif split == 'test':
        features = np.load(os.path.join(base_dir, 'feature_test.npy'))
        return None, features

def evaluate(w, b, age, features):
    w      = np.array(w).flatten()
    b      = float(np.array(b).flatten()[0])
    pred = features @ w + b
    loss = float(np.mean(np.abs(pred - age)))
    return loss, pred

def test(w, b, features, filename):
    w      = np.array(w).flatten()
    b      = float(np.array(b).flatten()[0])
    pred = features @ w + b
    np.savetxt(filename, pred)
    return pred

def load_image(file_names, size=(32, 32)):
    images, reduced, labels = [], [], []
    for fname in sorted(file_names):
```

```

    img = Image.open(fname).convert('RGBA')
    images.append(np.array(img))
    gray = img.convert('L').resize(size, Image.LANCZOS)
    reduced.append(np.array(gray, dtype=float) / 255.0)
    labels.append(1.0 if 'train_smile' in fname else -1.0)
return np.array(reduced), images, np.array(labels)

def visualize_results(images, preds, labels, ax=None):
    preds_flat = np.array(preds).flatten()
    y_hat = np.sign(preds_flat)
    n = len(images)
    fig, axes = plt.subplots(2, n, figsize=(2*n, 4))
    for i in range(n):
        sign_val = int(y_hat[i]) if y_hat[i] != 0 else 1
        axes[0, i].text(0.5, 0.5, str(sign_val),
                        ha='center', va='center', fontsize=14)
        axes[0, i].set_xlim(0,1); axes[0, i].set_ylim(0,1)
        axes[0, i].axis('off')
        axes[1, i].imshow(images[i]); axes[1, i].axis('off')
    plt.tight_layout()
    try:
        from IPython.display import display, clear_output
        clear_output(wait=True); display(fig); plt.close(fig)
    except ImportError:
        plt.show()

```

A.2 LinearRegression-publish.ipynb (核心函数)

```

# closed_form_solution

def closed_form_solution(age, features):
    H = np.column_stack((np.ones(len(features)), features))
    Y = age
    weights = np.linalg.lstsq(H, Y, rcond=None)[0]
    bias = weights[0]
    weights = weights[1:]
    return weights, bias

# gradient_descent

```

```
def gradient_descent(age, feature):
    assert len(age) == len(feature)
    np.random.seed(0)
    weights = np.random.randn(2048, 1)
    bias     = np.random.randn(1, 1)
    lr = 10e-3; n = len(age)
    for e in range(epoch):
        pred     = feature @ weights + bias
        diff     = pred - age.reshape(-1, 1)
        loss     = float(np.mean(diff ** 2))
        grad_w   = feature.T @ diff / n
        grad_b   = float(diff.mean())
        weights  = weights - lr * grad_w
        bias     = bias     - lr * grad_b
    return weights, bias

# stochastic_gradient_descent

def stochastic_gradient_descent(age, feature):
    assert len(age) == len(feature)
    np.random.seed(0)
    weights     = np.random.rand(2048, 1)
    bias        = np.random.rand(1, 1)
    lr          = 10e-5; batch_size = 16
    t           = len(age) // batch_size; loss = 0.0
    for e in range(epoch_sgd):
        n = np.random.permutation(len(feature))
        for m in range(t):
            bf = feature[n[m*batch_size:(m+1)*batch_size]]
            ba = age      [n[m*batch_size:(m+1)*batch_size]]
            pred     = bf @ weights + bias
            diff     = pred - ba.reshape(-1, 1)
            loss     = float(np.mean(diff ** 2))
            grad_w   = bf.T @ diff / len(ba)
            grad_b   = float(diff.mean())
            weights  = weights - lr * grad_w
            bias     = bias     - lr * grad_b
        print('=> epoch:', e+1, ' Loss:', round(loss, 4))
    return weights, bias
```


A.3 LinearPerceptron-publish.ipynb (感知机类)

```
class PrimalPerceptron(object):
    def __init__(self, x, y, w=None, b=None):
        num_sample, num_dims = x.shape
        np.random.seed(0)
        if w is None:
            w = np.random.randn(num_dims)
        if b is None:
            b = np.random.randn()
        self.x, self.y, self.w, self.b = x, y, w, b
        self.lr = 0.1

    def predict(self):
        preds = self.x @ self.w + self.b      # (n,) 线性得分
        y_hat = np.sign(preds)               # (n,) 分类标签
        return preds, y_hat

    def update(self):
        preds, y_hat = self.predict()
        mask = (y_hat * self.y) <= 0          # 误分类掩码
        if mask.sum() > 0:
            self.w += self.lr * (self.x[mask].T @ self.y[mask])
            self.b += self.lr * self.y[mask].sum()
        return
```